SDU

Pointers
Run to Complete Scheduling
SPI

Course outline

- **The plan is indicative, subject to change!**

Date	Lecture	Subject(s)	Lab	Assignment due
Feb 2	1	Introduction; ARM Cortex-M4	Lab1: Setting up the development env.	
Feb 9	2	EMP coding standard; Bit manipulation	Lab2: Bit manipulation	
Feb 16	3	State machines; The compiler chain; EMP-Board	Lab3: State machines	PF1 – Assignment 1
Feb 23	4	The task model; The pre-processor	Lab4: A clock radio task model	PF1 – Assignment 2
Mar 2	5	Queues and semaphores; Debugging	Lab5: Debug with a serial connection	PF1 – Assignment 3
Mar 9	6	Run to complete scheduler	Lab6: RTCS, Run to compile scheduler	PF1 – Assignment 4
Mar 16	7	More debugging; C: printf()	Lab7: RTCS, Debugger	PF1 – Assignment 5
March 23	8	FreeRTOS	Lab8: FreeRTOS	PF1 – Assignment 6
March 30 April 6		Easter holiday		
Apr 13	9	More queues; Assembler in C	Lab9: FreeRTOS (continued)	PF1 – Assignment 7
Apr 20	10	Re-entrance		PF1 – Assignment 8
Apr 27	11	Work on the final assignment, consultations		
May 4	12	Poster session		PF2- Final Assignment

Today’s Schedule

- Assignment04 presentation
- Quizz
- Recap
- Lecture
  - Pointers
  - RTCS – Run to Complete Scheduling
  - Teacher’s solution to A04
  - SPI
- Course midterm feedback
- Assignment05 unboxing
- Lab06

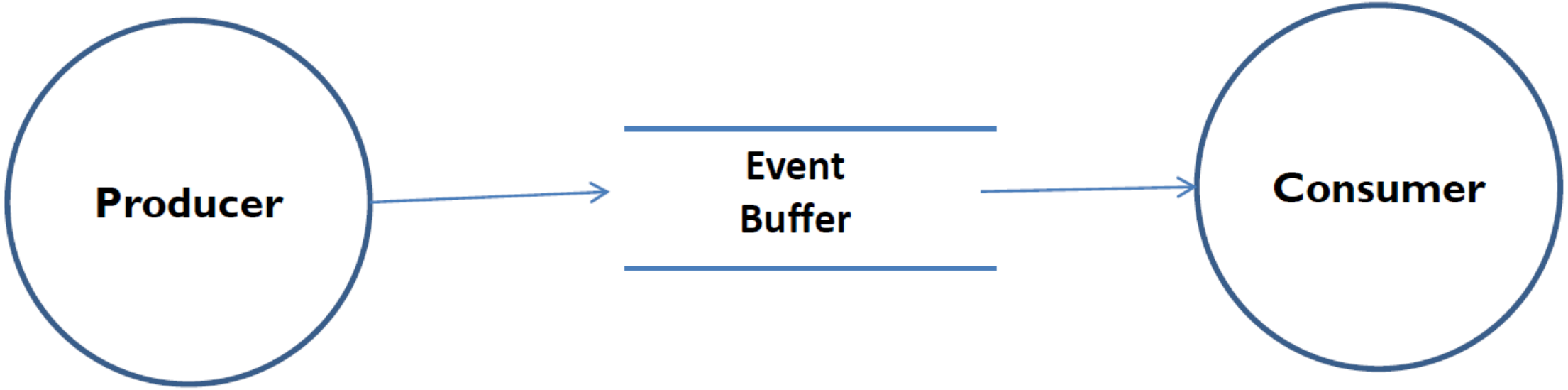
SDU

Quiz

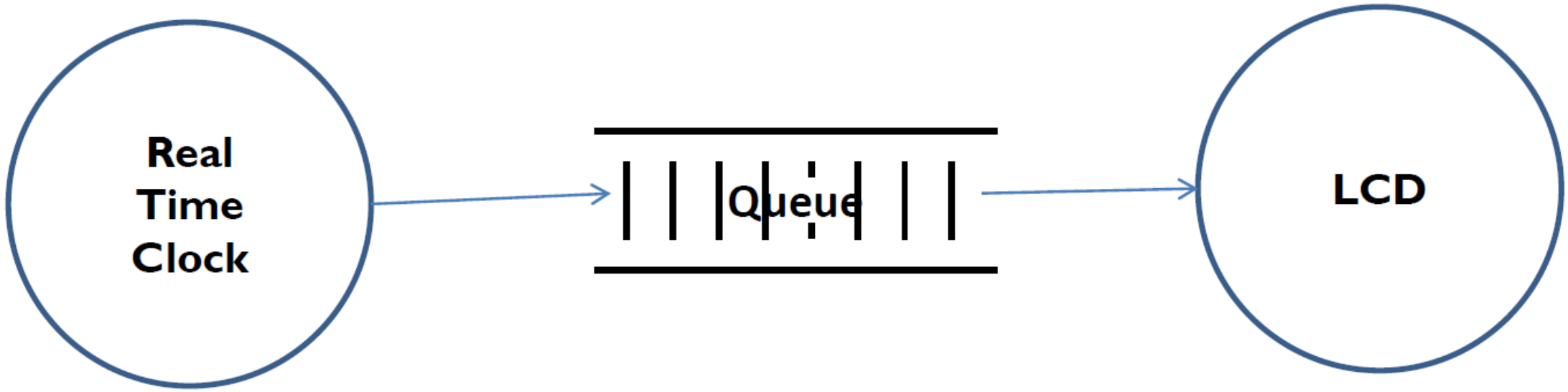
- slido.com
- #800423

Recap

Producer – consumer model



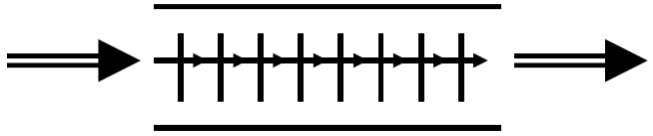
Producer – consumer model



”12:31:55”
100nS/char

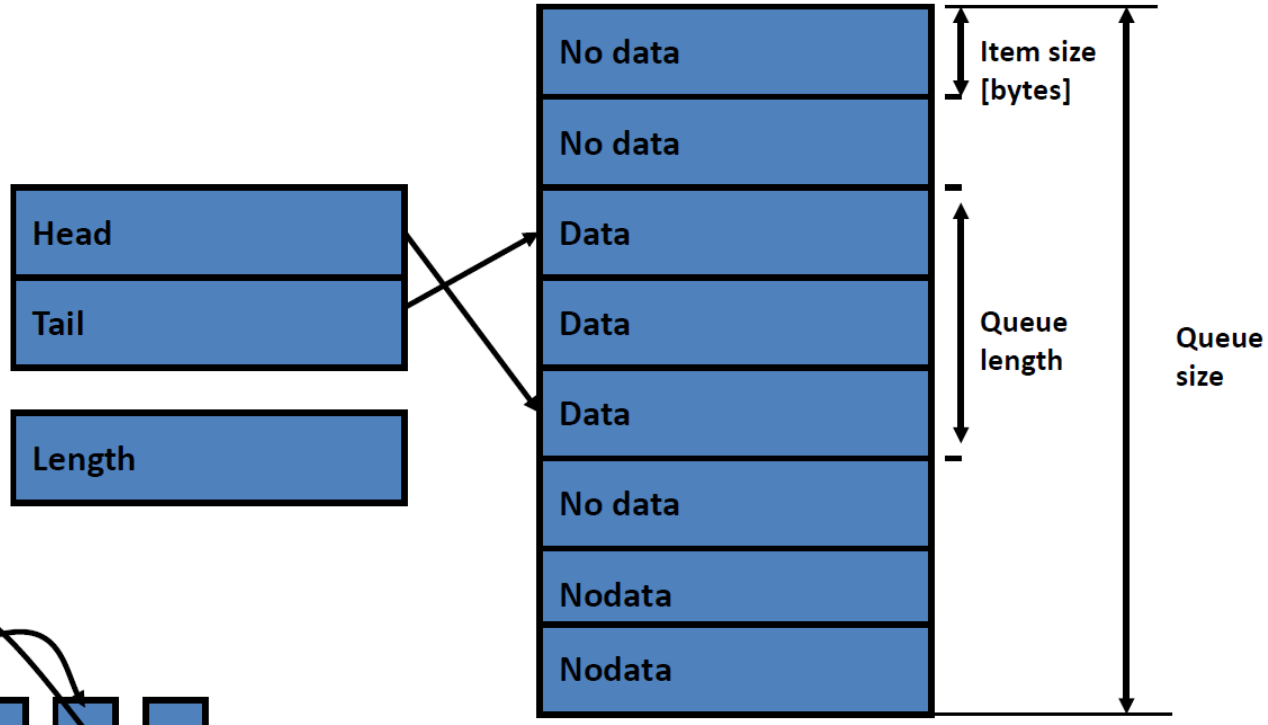
”12:31:55”
1mS/char

Queue implementation

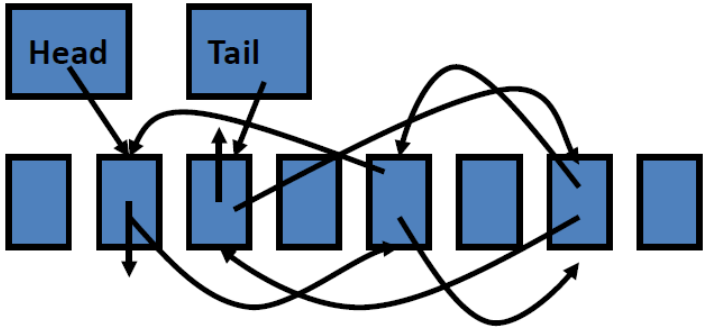


1) Move data through the queue (like supermarket)

2) Use pointers (pull a number
-like at the post office)

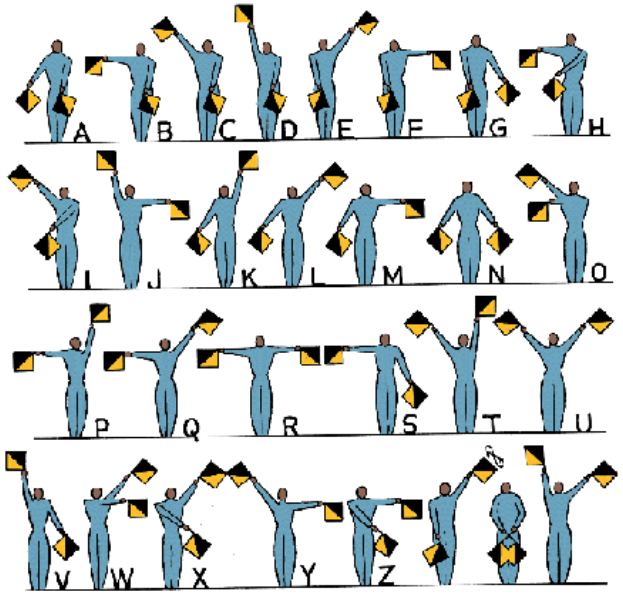


3) Linked List (like exam
– call the next)



SDU

Semaphores



- In real life: visual signaling with arms & flags
- In software: data structure for synchronization
- It's an integer, but
  - The only allowed operations are increment and decrement by one
  - When a task decrements the semaphore, if the result is negative, the task blocks itself and cannot continue until another task increments the semaphore
  - When a task increments the semaphore, if there are other tasks waiting, one of the waiting tasks gets unblocked.
- Counting or binary
- Example parking access control

Semaphore - signaling

- Making sure that statements in different tasks/threads execute in a specific order - serialization

Task A

```
Statement a1
Semaphore.signal()
```

Task B

```
Semaphore.wait()
Statement b1
```

SDU

Semaphore - mutex

- Secures mutually exclusive blocks to prevent concurrent access to the same resource
- Binary semaphore
- Protects ~~Creates~~ critical sections

Task A – clock update

```
mutex.wait()
        #critical section
        update(clock)
mutex.signal()
```

Task B – clock display

```
mutex.wait()
        #critical section
        display(clock)
mutex.signal()
```

Example – contentious meetings with duckie

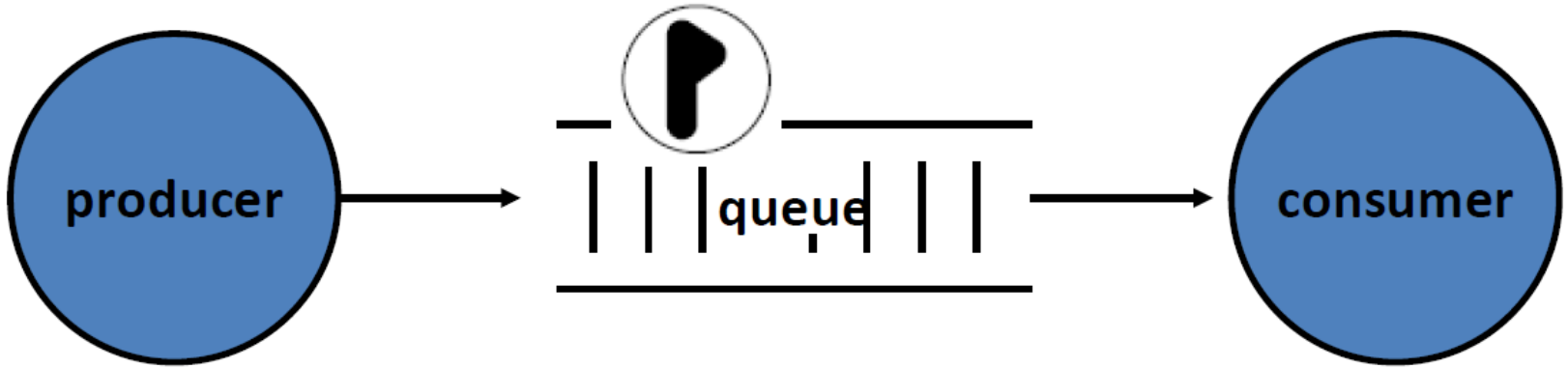
SDU

Binary semaphore vs. mutex

- Counting semaphore – multiple instances of a resource available – parking spots
- Mutex a version of a binary semaphore
  - A mutex can only be released by the task which acquired it
  - A binary semaphore can be released also by tasks with higher priority

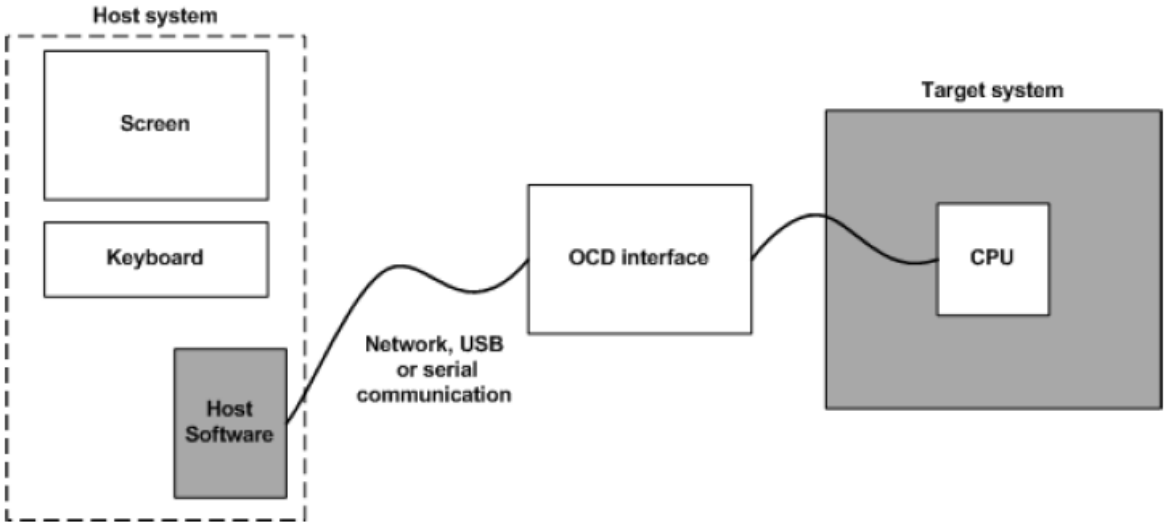
Protection of queues

Because of that, the semaphore is often built into the queue or closely connected to the queue.



ICD (In-Circuit Debugger) or OCD (On-Chip-debugger)

- Debug HW kernel,build into the CPU.
- Consumes no RAMor ROM.
- Hardware interface: 4-10 pins
  - JTAG(Joint TestAction Group), IEEEstandard
  - BDM(BackgroundDebug Monitor), Motorola



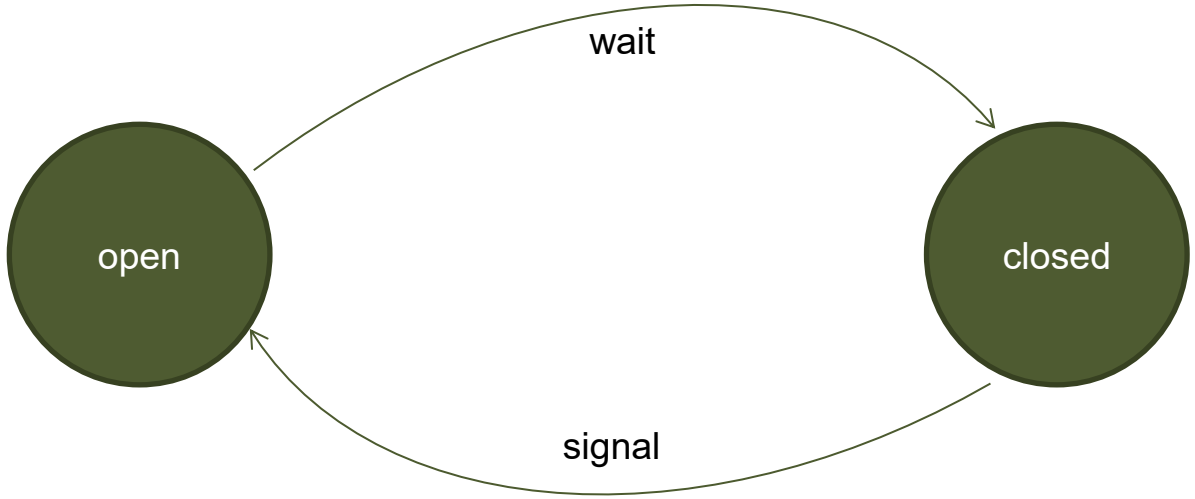
Semaphore purposes

- Signaling - serialization
- Mutex – mutual exclusion

Semaphore functions

Increment() ⇔ signal() ⇔ V()
Decrement() ⇔ wait() ⇔ P()

increment_and_wake_a_waiting_process_if_any ()
decrement_and_block_if_the_result_is_negative ()



SDU

Pointers - refresher

Just a refresher

- For intro to pointers see intro to C books or C tutorials – online lessons:
  - https://www.programiz.com/c-programming/c-pointers
  - https://www.w3resource.com/c-programming-exercises/

SDU

Variables and Pointers

```
INT16S var;
INT16S *ptr // pointer to INT16S
var = 6;
ptr = &var; // ptr = adr of var
*ptr = 8; // what ptr is pointing to
```

	value	address
	FFFF	0x80AC
var	0006	0x80AB
	FFFF	0x80AA
	FFFF	0x80A9
ptr	80AB	0x80A8
	FFFF	0x80A7
	FFFF	0x80A6

Special Pointers

```
ptr = NULL; // The NULL pointer

void *ptr; // The void pointer
```

- NULL pointer
  - To initialize a pointer without a memory address
  - To pass a null pointer as a function argument when no address should be passed
  - To check a pointer before accessing its stored value as address
- Void pointer
  - Can receive the address of a variable of any type

SDU

Operations with pointers

```
INT16S var1;
INT16S var2;
INT16S *ptr1;
INT16S *ptr2;
ptr1 = &var1;
*ptr1++;
(*ptr1)++;
ptr1++;
ptr1+=12;
ptr1 = &var1;
ptr2 = &var2;
var1 = 9;
*prt2 = *ptr1;
```


SDU

Pointers to pointers

```
INT16S var1;
INT16S var2;
INT16S *ptr;
INT16S **ptr_ptr;

var1 = 8;
ptr = &var1;
ptr_ptr = &ptr;

*ptr = 6;
var2 = **ptr_ptr;
```

Precedence	Operator	Description
1	++ --	Suffix/postfix increment and decrement
	( )	Function call
	[ ]	Array subscripting
	.	Structure and union member access
	->	Structure and union member access through pointer
	(*type*) { *list* }	Compound literal(C99)
2	++ --	Prefix increment and decrement[note 1]
	+ -	Unary plus and minus
	! ~	Logical NOT and bitwise NOT
	(*type*)	Cast
	*	Indirection (dereference)
	&	Address-of
	sizeof	Size-of[note 2]
	_Alignof	Alignment requirement(C11)

`*p++` is parsed as `*(p++)`, and not as `(*p)++`

Arrays

```
INT8U states[8];

for( i = 0; i < 8; i++ )
      states[i] = i + 1;

INT8U *p_states;
p_states = states; // Notice: No '&'

p_state += 2;
*p_state = 9;
states[6] = *p_state++;
(*p_state)++;
p_state++;
*p_state = state[4];
state[6] = *p_state;
```

Precedence	Operator	Description
1	++ --	Suffix/postfix increment and decrement
	( )	Function call
	[ ]	Array subscripting
	.	Structure and union member access
	->	Structure and union member access through pointer
	(*type*){*list*}	Compound literal(C99)
2	++ --	Prefix increment and decrement[note 1]
	+ -	Unary plus and minus
	! ~	Logical NOT and bitwise NOT
	(*type*)	Cast
	*	Indirection (dereference)
	&	Address-of
	sizeof	Size-of[note 2]
	_Alignof	Alignment requirement(C11)

Strings

- In C a string is a line of characters pointed to by a pointer and continuing until a '\0' character is found.
- The string might exist in memory of an array:

```
INT8U states[128];
```

- Or as a constant:

```
INT8U *p_str = "R2-D2";
string = p_str; // string = "R2-D2"
string = p_str + 3  ; // string = "D2"
```


SDU

structs

```
typedef struct
{
      INT8U LSB;
      INT8U MSB;
} t_ two_bytes;

t_two_bytes my_bytes;
my_bytes.LSB = 0x56;
t_two_bytes *p_bytes;
p_bytes = &my_bytes;
byte = p_bytes->LSB;
```

Pointers to functions

```
int (*p_funk)(int,int);
if( wanted_task == TASK_1 )
      p_funk = task_1;
else
      p_funk = task_2;
result = (*p_task)(1,100);
```

An example: tokenization

```
unsigned char *Token( pStr, Seperator )
unsigned char *pStr;
unsigned char Seperator;
{
      static unsigned char *pNext;
      pNext = pStr;
      while( (*pNext) && ( *pNext != Seperator ) )
            pNext++;
      if( *pNext )
            *pNext++ = '\0';
      return( pStr );
}
```

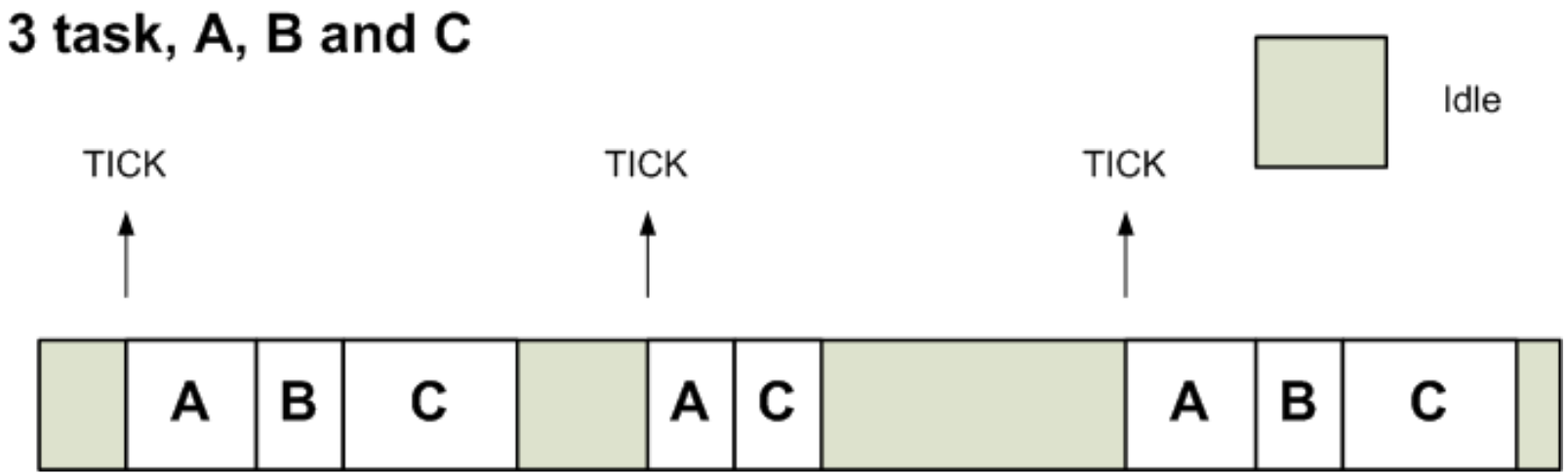
Task Scheduling

- Which task will get the CPU next?
- When to deprive the CPU from the task?
  - Pre-emptive (synonyms: defensive, precautionary, protective)
    - Task switch controlled by system
  - Non-preemptive
    - Depends on co-operation of tasks
    - Hope that tasks will end
    - Run to Complete Scheduler (RTCS)

SDU

Run to Complete Scheduling

- Each task runs until it is finished its job
- Scheduling is based on the tick
- The tick is the basic unit of time for task scheduling
- An example with 3 tasks
- What happens if C takes too long?



RTCS details

- RTCS is a very simple non-preemptive scheduler
- It features
  - Tasks
  - Task States (State Machines)
  - Task Events
  - Timers
  - Semaphores
  - Queues

SDU

RTCS – TCB – Task Control Block

```
typedef struct
{
      INT8U condition;
      INT8U name;
      INT8U state;
      INT8U event;
      INT8U sem;
      INT16U timer;
      void (*tf)(INT8U, INT8U, INT8U, INT8U); // Pointer to task function
} tcb;
```

API – Application programming interface

- a set of functions and procedures allowing the creation of applications that access the features or data of an operating system, application, or other service.

RTCS – Scheduler API

```
extern INT8U init_rtcs();
void schedule();
```


SDU

RTCS – Task API

```
extern HANDLE start_task( INT8U, void (*tf)(INT8U, INT8U, INT8U, INT8U) );
/*****************************************************************************
* Input : Handle (name/id) of the task.
* The task Function
* Output : Returns always 0.
* Function : Start new task.
*****************************************************************************/
```

RTCS – Timer and Semaphore API

```
extern void wait( INT16U );
/*********************************************************************
* Input : Timeout
* Output : -
* Function : Test function
*********************************************************************/
extern BOOLEAN wait_sem( INT8U, INT16U );
/*********************************************************************
* Input : Semaphore
* Timeout
* Output : TRUE if READY
* FALSE if WAITING
* Function : Wait for semaphore
*********************************************************************/
extern void signal( INT8U );
/*********************************************************************
* Input : Semaphore
* Output : -
* Function : Test function
*********************************************************************/
```

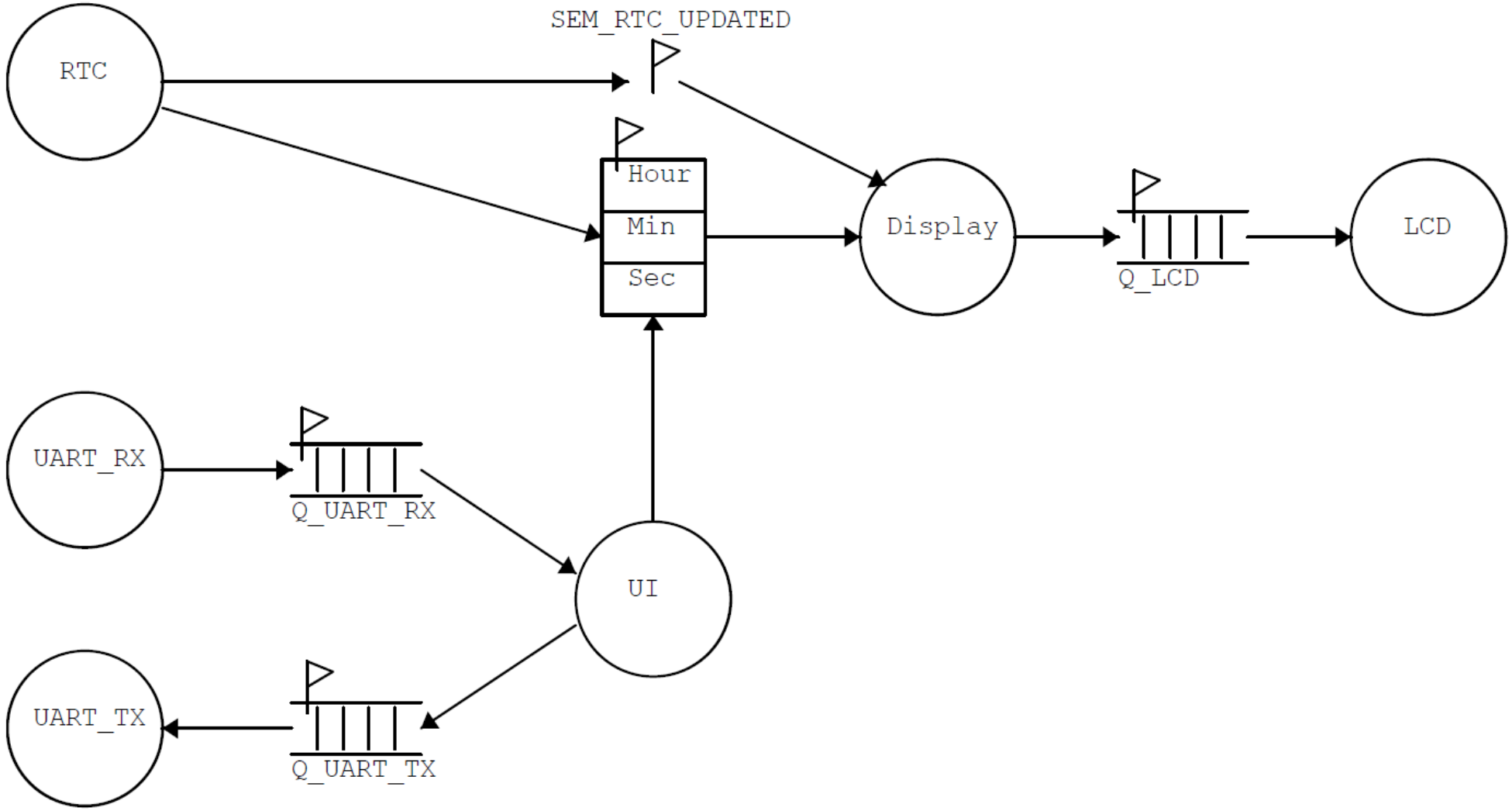
RTCS – Queue API

```
open_queue( INT8U );
/*************************************************************************
* Input : Queue name
* Output : Handle to queue
* -1 if queue is not opend
* Function : Open queue
*************************************************************************/
BOOLEAN put_queue( INT8U, INT8U, INT16U );
/*************************************************************************
* Input : Handle to Queue
* Character to put in Queue
* Timeout
* Output : TRUE if data produced
* FALSE if waiting for space in queue
* Function : produce data to queue
*************************************************************************/
BOOLEAN get_queue( INT8U, INT8U*, INT16U );
/*************************************************************************
* Input : handle to queue
* Pointer to databuffer
* Timeout
* Output : TRUE if data consumen
* FALSE if waiting for data
* Function : consume data from queue
*************************************************************************/
```

RTCS – Task Call-Back

```
void my_task( INT8U my_id, INT8U my_state, INT8U event, INT8U data )
/*****************************************************************
* Input : Handle (id) of the task.
* Current State of task
* Event
* Data (currently not used)
* Output : -
* Function : The task call-back.
*****************************************************************/
```

Assignment 4 – task diagram



Assignment 4

```
int main(void)
{
        init_gpio();
        init_rtcs();

        open_queue( Q_UART_TX );
        open_queue( Q_UART_RX );
        open_queue( Q_LCD );

        start_task( TASK_UART_TX, uart_tx_task );
        start_task( TASK_UART_RX, uart_rx_task );
        start_task( TASK_UI, ui_task );
        start_task( TASK_LCD, lcd_task );
        start_task( TASK_RTC, rtc_task );
        start_task( TASK_DISPLAY_RTC, display_rtc_task );

        schedule();

        return(0);
}
```

SPI

Serial Peripheral Interface (SPI)

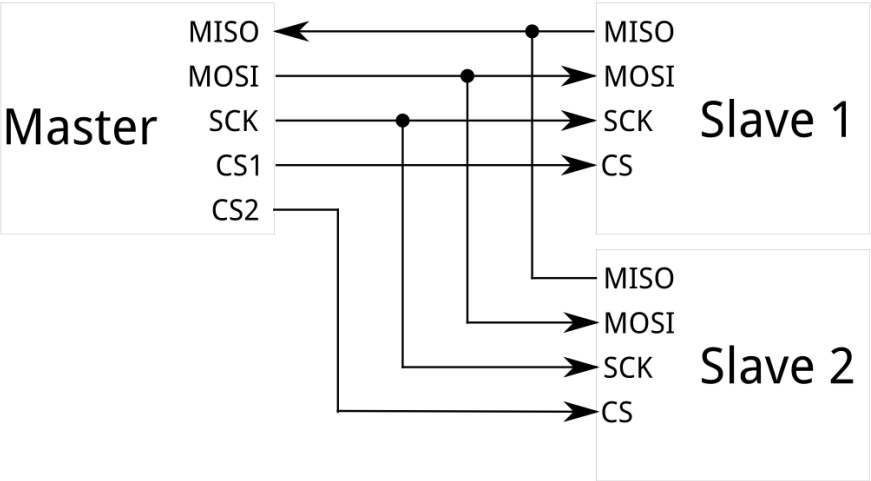
- A synchronous full-duplex serial communication interface
  - Synchronous – clock is transmitted
  - Full-duplex – sending and receiving on different lines
- Four wire interface
- Close proximity communication (<10m) – ideal for embedded
- Developed my Motorola in the mid 1980.
- De facto standard other than IIC
- Single Master, multiple slave configuration

Serial Peripheral Interface (SPI)

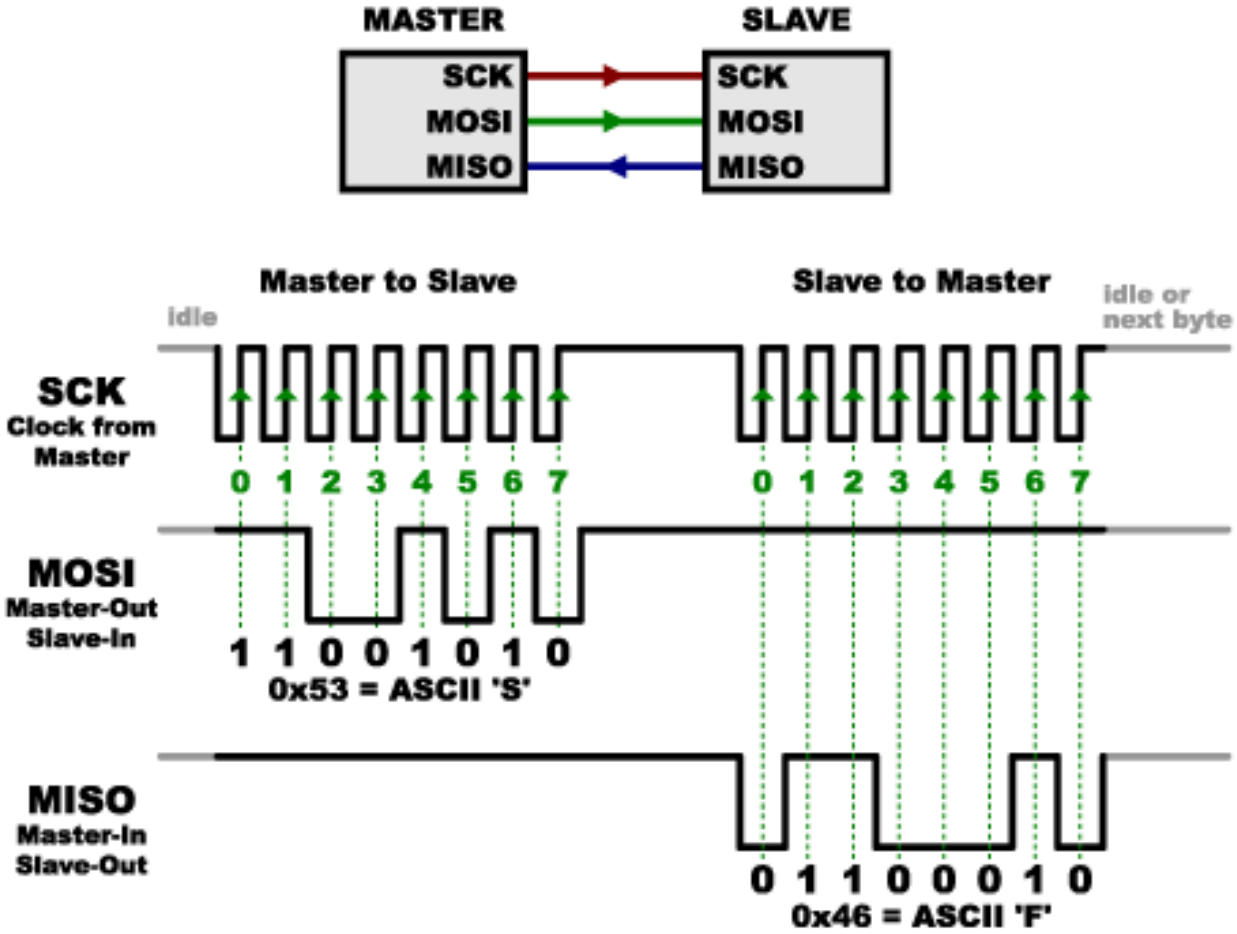
- Compared to UART
  - UART is asynchronous – no clock is transmitted, a speed needs to be agreed upon before the start of the communication
  - SPI – synchronous – clock set by master
- Compared to IIC
  - IIC is a two-wire half-duplex synchronous communication interface
    - It either sends or receives
    - Multi master, multi slave
  - SPI is four wire, much faster
    - Single master

Serial Peripheral Interface (SPI)

- MISO – master input, slave output
- MOSI – master output, slave input
- SCK – serial clock
- CS1, CS2 – chip select



Serial Peripheral Interface (SPI)



SPI on TM4C123 microcontroller

- The SSI (synchronous serial interface) ports need to be used

Pin Name	Pin Number	Pin Mux / Pin Assignment	Pin Type	Buffer Type[a]	Description
SSI0Clk	19	PA2 (2)	I/O	TTL	SSI module 0 clock
SSI0Fss	20	PA3 (2)	I/O	TTL	SSI module 0 frame signal
SSI0Rx	21	PA4 (2)	I	TTL	SSI module 0 receive
SSI0Tx	22	PA5 (2)	O	TTL	SSI module 0 transmit
SSI1Clk	30 61	PF2 (2) PD0 (2)	I/O	TTL	SSI module 1 clock.
SSI1Fss	31 62	PF3 (2) PD1 (2)	I/O	TTL	SSI module 1 frame signal.
SSI1Rx	28 63	PF0 (2) PD2 (2)	I	TTL	SSI module 1 receive.
SSI1Tx	29 64	PF1 (2) PD3 (2)	O	TTL	SSI module 1 transmit.
SSI2Clk	58	PB4 (2)	I/O	TTL	SSI module 2 clock.
SSI2Fss	57	PB5 (2)	I/O	TTL	SSI module 2 frame signal.
SSI2Rx	1	PB6 (2)	I	TTL	SSI module 2 receive.
SSI2Tx	4	PB7 (2)	O	TTL	SSI module 2 transmit.
SSI3Clk	61	PD0 (1)	I/O	TTL	SSI module 3 clock.
SSI3Fss	62	PD1 (1)	I/O	TTL	SSI module 3 frame signal.
SSI3Rx	63	PD2 (1)	I	TTL	SSI module 3 receive.
SSI3Tx	64	PD3 (1)	O	TTL	SSI module 3 transmit.

SPI on TM4C123 microcontroller

- The SSI (synchronous serial interface) ports need to be used
  - SCK > SSI0Clk
  - CS > SSI0Fss
  - MISO > SSI0Rx
  - MOSI > SSI0Tx

Pin Name	Pin Number	Pin Mux / Pin Assignment	Pin Type	Buffer Type[a]	Description
SSI0Clk	19	PA2 (2)	I/O	TTL	SSI module 0 clock
SSI0Fss	20	PA3 (2)	I/O	TTL	SSI module 0 frame signal
SSI0Rx	21	PA4 (2)	I	TTL	SSI module 0 receive
SSI0Tx	22	PA5 (2)	O	TTL	SSI module 0 transmit
SSI1Clk	30 61	PF2 (2) PD0 (2)	I/O	TTL	SSI module 1 clock.
SSI1Fss	31 62	PF3 (2) PD1 (2)	I/O	TTL	SSI module 1 frame signal.
SSI1Rx	28 63	PF0 (2) PD2 (2)	I	TTL	SSI module 1 receive.
SSI1Tx	29 64	PF1 (2) PD3 (2)	O	TTL	SSI module 1 transmit.
SSI2Clk	58	PB4 (2)	I/O	TTL	SSI module 2 clock.
SSI2Fss	57	PB5 (2)	I/O	TTL	SSI module 2 frame signal.
SSI2Rx	1	PB6 (2)	I	TTL	SSI module 2 receive.
SSI2Tx	4	PB7 (2)	O	TTL	SSI module 2 transmit.
SSI3Clk	61	PD0 (1)	I/O	TTL	SSI module 3 clock.
SSI3Fss	62	PD1 (1)	I/O	TTL	SSI module 3 frame signal.
SSI3Rx	63	PD2 (1)	I	TTL	SSI module 3 receive.
SSI3Tx	64	PD3 (1)	O	TTL	SSI module 3 transmit.

Midterm class feedback

Please fill out this anonymous form to provide feedback:
https://forms.gle/YNRsd2iXAxz9Qhcd8